

Guia de Referência Moderno: Git & GitHub

André D. Bueno

2026-01-31

Contents

1	Guia de Referência Moderno: Git & GitHub	2
2	Configuração Inicial & Autenticação	2
2.1	Identidade Básica	2
2.2	Baixar o git	2
2.3	Autenticação Moderna (SSH)	2
2.3.1	Passo 1: Gerar a nova chave SSH (Sem senha)	2
2.3.2	Passo 2: Registrar a nova chave no sistema operacional	2
2.3.3	Passo 3: Adicionar a chave pública ao seu perfil do GitHub	2
2.3.4	Passo 4: Atualizar o seu repositório local existente para usar SSH	3
2.3.5	Passo 5: Testar a conexão e fazer o envio (Push)	3
3	GitHub: Primeiros Passos	3
3.1	Criar Repositório (Do Zero)	3
3.2	Clonar um Repositório HTTPS Existente (Evite)	3
3.3	Clonar um Repositório Usando SSH (Sem usar HTTPS, Ideal)	3
3.4	Adicionar Colaboradores	3
4	Fluxo de Trabalho Diário	3
4.1	Verificar Estado (O comando mais importante)	3
4.2	Preparar (Staging)	4
4.3	Salvar (Commit)	4
4.4	Sincronizar (Sync)	4
5	Padrões de Nomes (Branches e Tags)	4
5.1	Prefixos Recomendados	4
5.2	O que NÃO fazer (Proibido)	4
5.3	Tags de Versão (Semantic Versioning)	4
6	Manipulação de Branches (Modernizado)	5
7	Correções, Ajustes e "Salva-Vidas"	5
7.1	Desfazer alterações locais (Antes do Commit)	5
7.2	"Guardar para depois" (Stash)	5
7.3	O Botão de Pânico (Reset)	5
8	Ferramentas e Interfaces Gráficas	5
8.1	Clientes Git (GUIs)	5
8.2	Plugins para Editores (Essenciais)	5
8.2.1	Para Alunos (VS Code)	5
8.2.2	Para o Professor (Emacs)	6
9	Arquivo .gitignore (Modelo Acadêmico)	6
10	Receitas Práticas (Exemplos)	6
10.1	1. Commit com Data Passada	6
10.2	2. Criar Tags de Versão (Releases)	6
10.3	3. Corrigir mensagem do último commit	6
10.4	4. Sincronizar um Fork (Upstream)	6

1 Guia de Referência Moderno: Git & GitHub

2 Configuração Inicial & Autenticação

Antes de começar, garanta que sua identidade e métodos de acesso ao github estão configurados.

2.1 Identidade Básica

Configura seu nome nas "etiquetas" dos commits.

```
git config --global user.name "Seu Nome"
git config --global user.email "seu_email@exemplo.com"
git config --global core.editor "emacs" # ou code, vim, nano
git config --global init.defaultBranch main # Padroniza 'main'
git config --global pull.rebase true # Evita merges desnecessários ao baixar
git config --global color.ui auto
```

2.2 Baixar o git

- Linux (Debian/Ubuntu): `sudo apt install git`
- Fedora: `sudo dnf install git`
- Windows/Mac: <http://git-scm.com>

2.3 Autenticação Moderna (SSH)

O GitHub não aceita mais senha de usuário no terminal. Use Chaves SSH.

Se o sistema pedir a *passphrase* e você não se lembrar dela, siga os passos abaixo para gerar uma chave limpa e configurar seu repositório local.

2.3.1 Passo 1: Gerar a nova chave SSH (Sem senha)

Abra o terminal e execute o comando:

```
ssh-keygen -t ed25519 -C "bueno@lenep.uenf.br"
```

Atenção às perguntas do terminal:

1. Enter file in which to save the key... -> Aperte **[Enter]**
2. Overwrite (y/n)? -> Digite **y** e aperte **[Enter]** (para sobrescrever a chave antiga)
3. Enter passphrase... -> Deixe em branco, apenas aperte **[Enter]**
4. Enter same passphrase again: -> Aperte **[Enter]** novamente

2.3.2 Passo 2: Registrar a nova chave no sistema operacional

Ative o agente do SSH e adicione a nova chave criada à memória do sistema:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

2.3.3 Passo 3: Adicionar a chave pública ao seu perfil do GitHub

1. Mostre o conteúdo da chave pública no terminal:

```
cat ~/.ssh/id_ed25519.pub
```

2. Copie todo o texto exibido (ele começa com `ssh-ed25519`).

3. Vá no navegador até o GitHub:

- Clique na sua **Foto de Perfil** (canto superior direito) -> **Settings**.
- Na barra lateral esquerda, clique em **SSH and GPG keys**.
- Remova a chave antiga (clcando em **Delete**).
- Clique em **New SSH Key**.
- Cole o texto copiado no campo **Key** e dê um título no campo **Title** (ex: PC Bueno).
- Clique em **Add SSH Key**.

2.3.4 Passo 4: Atualizar o seu repositório local existente para usar SSH

Se você já deu `git init` e adicionou o repositório via HTTPS, mude a URL local para apontar para o endereço SSH correto, exemplo:

```
Modelo: git remote set-url origin git@github.com:ldsc/NomeDoSeuProjeto.git
git remote set-url origin git@github.com:ldsc/ProjetoEngenharia-Didatico-03-Equipamentos-SistemaInterruptorLampada.git
```

2.3.5 Passo 5: Testar a conexão e fazer o envio (Push)

1. Teste se o GitHub reconhece a sua nova chave:

```
ssh -T git@github.com
```

A resposta deve ser: Hi andreduarte! You've successfully authenticated...

2. Faça o envio inicial do seu projeto sem que nenhuma senha seja solicitada:

```
git add .
git commit -m "mensagem informando o que atualizou"
git push -u origin main
```

3 GitHub: Primeiros Passos

3.1 Criar Repositório (Do Zero)

1. No site, clique em **New repository**.
2. **Dica:** Se for subir um código que já existe no seu PC, crie o repo **VAZIO** (sem README/gitignore).

3.2 Clonar um Repositório HTTPS Existente (Evite)

```
# HTTP
git clone https://github.com/ldsc/LDSC-Ajuda-DocumentsAuxiliares
# Modelo de projeto completo
ex:
git clone https://github.com/ldsc/LDSC-ProjetoEngenharia-2-Software-TituloProjeto-ModeloCompleto

# É possível pedir para o git memorizar suas credenciais
# Digite o seguinte comando para dizer ao Git para memorizar suas credenciais:
git config --global credential.helper store
PS: Na primeira vez que ele pedir o seu Usuário e Senha, digite-os. Atenção: O GitHub não aceita mais a senha
```

3.3 Clonar um Repositório Usando SSH (Sem usar HTTPS, Ideal)

```
# Para clonar repositórios sem precisar digitar usuário e senha (usando apenas a sua chave SSH), você deve se autenticar com a chave SSH.

# O comando padrão é:
git clone git@github.com:usuario/nome-do-repositorio.git

# Exemplo:
git clone git@github.com:ldsc/ProjetoEngenharia-Didatico-03-Equipamentos-SistemaInterruptorLampada.git
```

3.4 Adicionar Colaboradores

Settings -> Collaborators -> Add people (insira o email do aluno/colega).

4 Fluxo de Trabalho Diário

A rotina sagrada do desenvolvedor.

4.1 Verificar Estado (O comando mais importante)

```
git status # Mostra o que mudou e onde você está
```

4.2 Preparar (Staging)

```
git add .           # Adiciona TUDO (novos, modificados, deletados)
git add -u          # Adiciona apenas modificados (ignora arquivos novos)
```

4.3 Salvar (Commit)

```
git commit -m "verbo/ação: o que foi feito"
```

4.4 Sincronizar (Sync)

```
git pull origin main # Baixa atualizações
git push origin main # Envia seus commits
```

5 Padrões de Nomes (Branches e Tags)

Evite dores de cabeça seguindo convenções universais.

5.1 Prefixos Recomendados

Para criar branches organizadas (`git switch -c nome`):

Prefixo	Uso	Exemplo
:—	:—	:—
feat/	Nova funcionalidade	feat/nova-interface-grafica
fix/	Correção de erro (bug)	fix/correcao-calculo-imc
docs/	Apenas documentação	docs/atualiza-readme
refactor/	Melhoria de código (sem mudar função)	refactor/limpeza-classe-pessoa

5.2 O que NÃO fazer (Proibido)

1. **Espaços:** Jamais use espaços. Use hifens (`minha-branch`) ou underscores.

- corrigir erro | `fix/corrigir-erro`

2. **Acentos e Cedilha:** Evite caracteres latinos, pois terminais Windows/Linux podem divergir na codificação.

- correção-do-cabecalho | `fix/correcao-cabecalho`

3. **Nomes Vagos:**

- teste, temp, ajuste, seuNome

5.3 Tags de Versão (Semantic Versioning)

Use o padrão `vX.Y.Z`.

- `v1.0.0` (Major): Mudança que quebra compatibilidade.
- `v1.1.0` (Minor): Nova funcionalidade (compatível).
- `v1.1.1` (Patch): Correção de bug simples.

6 Manipulação de Branches (Modernizado)

Ação	Comando Moderno	Comando Antigo (Legacy)
:—	:—	:—
Criar e Mudar	<code>git switch -c feat/nome</code>	<code>git checkout -b feat/nome</code>
Mudar de Ramo	<code>git switch feat/nome</code>	<code>git checkout feat/nome</code>
Voltar p/ Main	<code>git switch main</code>	<code>git checkout master</code>
Deletar Ramo	<code>git branch -d feat/nome</code>	<code>git branch -d feat/nome</code>

7 Correções, Ajustes e "Salva-Vidas"

7.1 Desfazer alterações locais (Antes do Commit)

```
git restore arquivo.txt      # Descarta mudanças no arquivo
git restore .                # PERIGO: Descarta TUDO não salvo
```

7.2 "Guardar para depois" (Stash)

Útil para interrupções rápidas (ex: corrigir bug urgente na main).

```
git stash                    # Guarda trabalho incompleto
git switch main              # Muda de branch
# ...corrige bug...
git switch feat/minha-branch
git stash pop                # Recupera o trabalho
```

7.3 O Botão de Pânico (Reset)

```
# Destrói commits locais e deixa igual ao último ponto salvo
git reset --hard HEAD
```

8 Ferramentas e Interfaces Gráficas

Nem tudo precisa ser feito no terminal.

8.1 Clientes Git (GUIs)

Facilitam a visualização do histórico e resolução de conflitos.

1. **GitKraken:** (Linux/Win/Mac)

- Visualmente o mais bonito e poderoso. Gratuito para repos públicos.
- Site: <https://www.gitkraken.com>

2. **Gitg:** (Linux - GNOME)

- Simples e leve, ótimo para ver o histórico.
- Instalação: `sudo apt install gitg` ou `sudo dnf install gitg`

3. **Git Cola:** (Linux - FOSS)

- Interface rápida e funcional.
- Instalação: `sudo apt install git-cola`

8.2 Plugins para Editores (Essenciais)

8.2.1 Para Alunos (VS Code)

- **GitLens:**
 - O padrão de mercado. Mostra quem editou cada linha (Blame) direto no código e histórico visual.
- **Git Graph:**
 - Cria o desenho da árvore de commits dentro do VS Code.

8.2.2 Para o Professor (Emacs)

- **Magit:** É indiscutivelmente a melhor interface de Git já criada para qualquer editor.
 - Permite fazer stage de pedaços de código (hunks), rebase interativo e commit com poucos atalhos.
 - Instalação: via MELPA (M-x package-install RET magit).

9 Arquivo .gitignore (Modelo Acadêmico)

Crie este arquivo na raiz para ignorar lixo gerado por compiladores e editores.

```
# --- C++ ---
*.o *.obj *.exe *.out
bin/ build/
# --- LaTeX / LyX ---
*.aux *.log *.toc *.fls *.fdb_latexmk *.synctex.gz
*.pdf # Opcional: Se quiser gerar o PDF localmente mas não subir
*.lyx~ # Backup do LyX
# --- Editores ---
# Emacs
*~
\#*\#
.\#*
# VS Code
.vscode/
*.code-workspace
# --- Sistema ---
.DS_Store
Thumbs.db
```

10 Receitas Práticas (Exemplos)

10.1 1. Commit com Data Passada

```
git add .
git commit -m "Implementação do módulo X" --date="2026-01-20 14:00:00"
```

10.2 2. Criar Tags de Versão (Releases)

```
git tag -a v0.2.4 -m "Versão com correção do bug de login"
git push origin v0.2.4
```

10.3 3. Corrigir mensagem do último commit

```
git commit --amend -m "Nova mensagem correta"
# CUIDADO: Não faça isso se já deu push!
```

10.4 4. Sincronizar um Fork (Upstream)

Atualizar seu projeto com o original do professor.

```
git remote add upstream https://github.com/professor/projeto.git
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```